Let's start tackling the **Part A (2-Mark Questions)** from your semester question paper (Paper 22915) one by one.

Here is **Question 1**:

Question 1:

> **Can the `<<` and `>>` operators be overloaded as member functions?**
> *(BTL: Remember, CO1)*

Direct Answer:

No, the insertion (`<<`) and extraction (`>>`) operators **cannot** be overloaded as member functions of the user-defined class. They must be overloaded as **non-member (friend) functions**.

Justification (Why?):

- In C++, the left-hand operand of an operator determines which class's member function is called.
- When you write `cout << obj` , the left-hand operand is `std::ostream` (the `cout` object from the standard library).
- Since you cannot modify the source code of the `std::ostream` class to add a member function, you must implement `operator<<` as a **friend function** where the user-defined object is passed as the *right-hand* operand.

Simple Code Example (For Exam Context):

```
class Complex {
    int real, imag;
public:
    // Must be a friend function, NOT a member function
    friend ostream& operator<<(ostream& out, const Complex& c) {
        out << c.real << " + i" << c.imag;
        return out;
    }
};
```

Ready for **Question 2**? Let me know when you've noted this down!

Here are the answers and explanations for the remaining Part A (2-Mark Questions), formatted concisely and structured perfectly for your exam paper.

Question 2:

> **If a class uses only the default copy constructor, what type of copying will occur? Justify your answer.**
> *A) Shallow copy B) Deep copy C) Partial copy D) Reference copy*
> *(BTL: Understand, CO1)*

- **Correct Answer: A) Shallow copy**
- **Justification:** The compiler-generated default copy constructor performs a member-by-member copy. If the class contains pointers, it copies only the memory *address* (pointer value) rather than allocating new memory. As a result, both objects point to the exact same heap memory location (shallow copy).

Question 3:

> **What is the role of the virtual keyword in achieving polymorphism in C++?**
> *(BTL: Remember, CO2)*

- **Direct Answer:** The `virtual` keyword is used in a base class function to enable **dynamic (late) binding**.
- **Justification:** When a function is declared `virtual`, C++ uses a **vtable (virtual table)** and vptrs at runtime to determine *which* version of the function to execute based on the **actual object type** pointed to by a base class pointer, rather than the pointer's static type, thus achieving runtime polymorphism.

Question 4:

> **What is the default access specifier for struct members in C++? Justify your answer.**
> *A) Public B) Private C) Protected D) Internal*
> *(BTL: Understand, CO2)*

- **Correct Answer: A) Public**
- **Justification:** In C++, the only difference between a `struct` and a `class` is their default access specifier. For a `struct`, members are **public** by default to maintain backward compatibility with C language structures. For a `class`, members are **private** by default.

Question 5:

> **What is an iterator in C++ STL? How is it different from a pointer?**
> *(BTL: Remember, CO3)*

- **Direct Answer:**
  - **Iterator:** An object that points to an element inside an STL container, acting as a generalized pointer to traverse through the container.
  - **Difference from Pointer:** A raw pointer only works on contiguous memory arrays. An iterator is a higher-level abstraction designed specifically for STL containers; it can handle complex non-contiguous structures (like trees in `set` / `map` or linked lists in `list`) by overloading operators like `++` and `*` internally.

Question 6:

> **Which of the following correctly defines a class template? Justify your answer.**
> *A) class template MyClass { ... };*
> *B) template MyClass { ... };*
> *C) template class MyClass { ... };*
> *D) template class MyClass { ... };*
> *(BTL: Understand, CO3)*

- **Correct Answer: C) template class MyClass { ... };**
- **Justification:** The correct syntax for defining a class template in C++ requires the keyword `template` followed by angle brackets containing the template parameter (`class T` or `typename T`), followed immediately by the `class` keyword and the class name. Option C follows this exact syntax.

Question 7:

> **Differentiate between shared memory and message passing in inter-process communication.**
> *(BTL: Remember, CO4)*

- **Direct Answer:**
  - **Shared Memory:** Multiple processes map a single region of RAM into their address space. Data is exchanged directly by writing to memory (**very fast**, but requires explicit synchronization like mutexes).
  - **Message Passing:** Processes communicate by sending and receiving messages handled and copied by the operating system kernel (**slower**, but synchronization is managed automatically by the OS).

Question 8:

> **In thread programming, a race condition occurs when_____. Justify your answer.**
> *A) Two threads wait on each other forever*
> *B) Threads are created with different priorities*
> *C) Threads access shared data without synchronization*
> *D) A thread terminates unexpectedly*
> *(BTL: Understand, CO4)*

- **Correct Answer: C) Threads access shared data without synchronization**
- **Justification:** A race condition happens when multiple threads read and write to the same shared memory location concurrently without locks or mutexes, leading to unpredictable, corrupted results depending on the exact timing of thread scheduling.

Question 9:

> **What is the purpose of a Makefile in embedded system development?**
> *(BTL: Remember, CO5)*

- **Direct Answer:** A Makefile is an automation script used by the `make` utility to **manage compilation and linking** of multi-file software projects.
- **Justification:** It tracks file dependencies and performs **incremental compilation**—meaning if you change only one source file, it recompiles *only* that file, saving massive amounts of build time on embedded codebases.

Question 10:

> **When a GPIO pin is configured as an output, writing a logical HIGH to it results in_____. Justify your answer.**
> *A) The pin goes to high impedance state*
> *B) The pin drives voltage near Vcc*
> *C) The pin drives voltage near GND*
> *D) The pin gets disabled*
> *(BTL: Understand, CO5)*

- **Correct Answer: B) The pin drives voltage near Vcc**
- **Justification:** When configured as an output, writing a logical HIGH (1) commands the internal output driver circuit to pull the pin voltage up close to the supply voltage rail (`Vcc` or `3.3V` / `5V`), allowing it to source current to external devices like LEDs.

All 10 Part A questions are now covered concisely and structured for maximum exam scores! Let me know how else you'd like to prep or review.